

Computer System Architecture

Introduction

A computer system can be organized in a number of different ways, which we can categorize roughly according to the number of general-purpose processors used.

1. Single-Processor Systems
2. Multiprocessor Systems
3. Clustered Systems

Single-Processor Systems

Most computer systems used a single processor containing one CPU with a single processing core.

These systems may have **device-specific processors**.

All of these special-purpose processors **run a limited instruction set** and do not run processes. Sometimes, they are managed by the operating system, in that the operating system sends them information about their next task and monitors their status.

For example, a disk-controller microprocessor receives a sequence of requests from the main CPU core and implements its own disk queue and scheduling algorithm.

Multi-Processor Systems

Systems have two (or more) processors, each with a single-core CPU

The processors share the computer bus and sometimes the clock, memory, and peripheral devices.

The primary advantage of multiprocessor systems is **increased throughput**.

Multiprocessor systems can **cost less** than equivalent multiple single-processor systems

The most common multiprocessor systems use **symmetric multiprocessing (SMP)**, in which each peer CPU processor performs all tasks, including operating-system functions and user processes.

Multi-Processor Systems

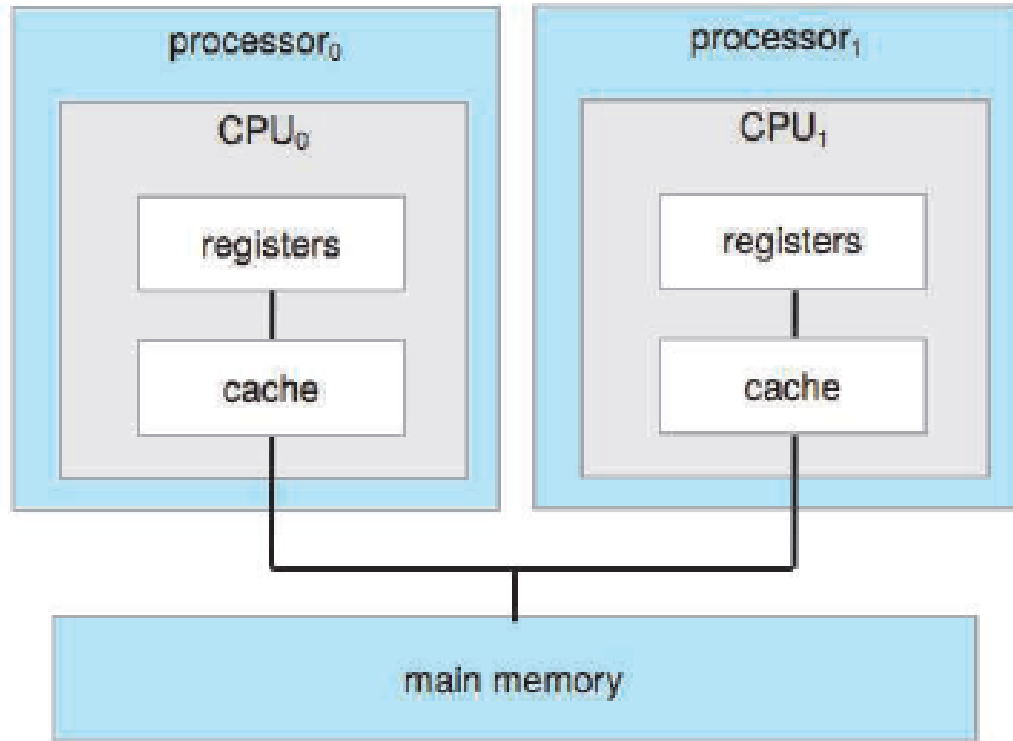


Figure: Symmetric multiprocessing architecture.

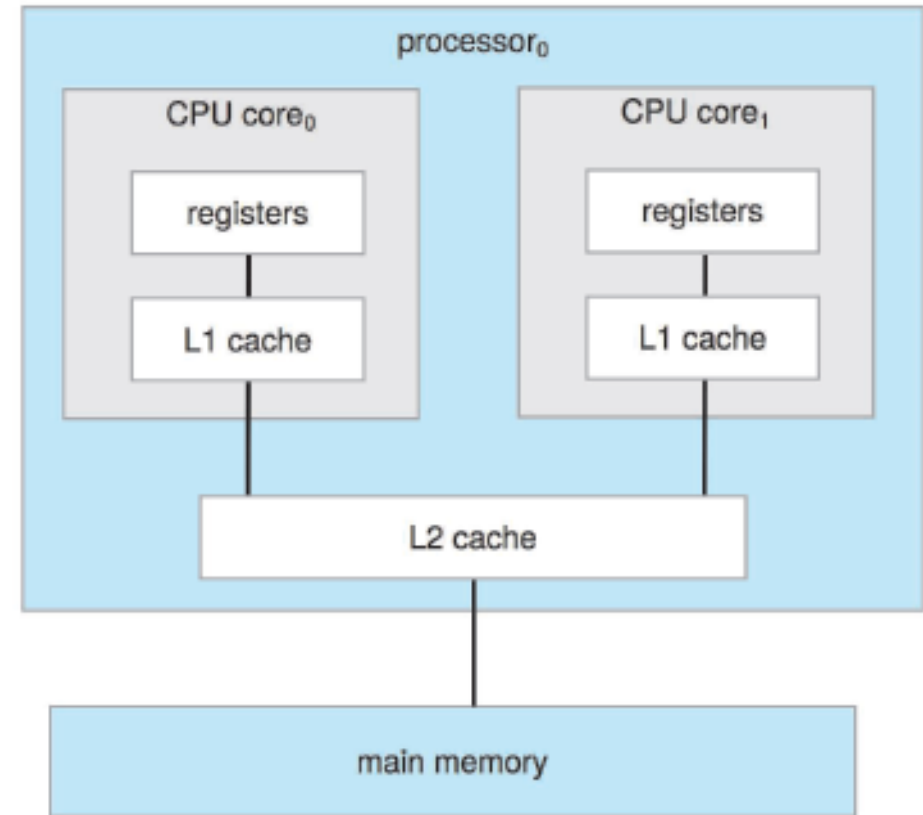


Figure: A dual-core design with two cores on the same chip.

Multi-Processor Systems

Multicore systems - multiple computing cores reside on a single chip.

Multicore systems can be more efficient than multiple chips with single cores because on-chip communication is faster than between-chip communication

In addition, one chip with multiple cores uses significantly less power than multiple single-core chips.

Adding additional CPUs to a multiprocessor system will increase computing power; however, it does not scale very well, and once we add too many CPUs, contention for the system bus becomes a bottleneck and performance begins to degrade.

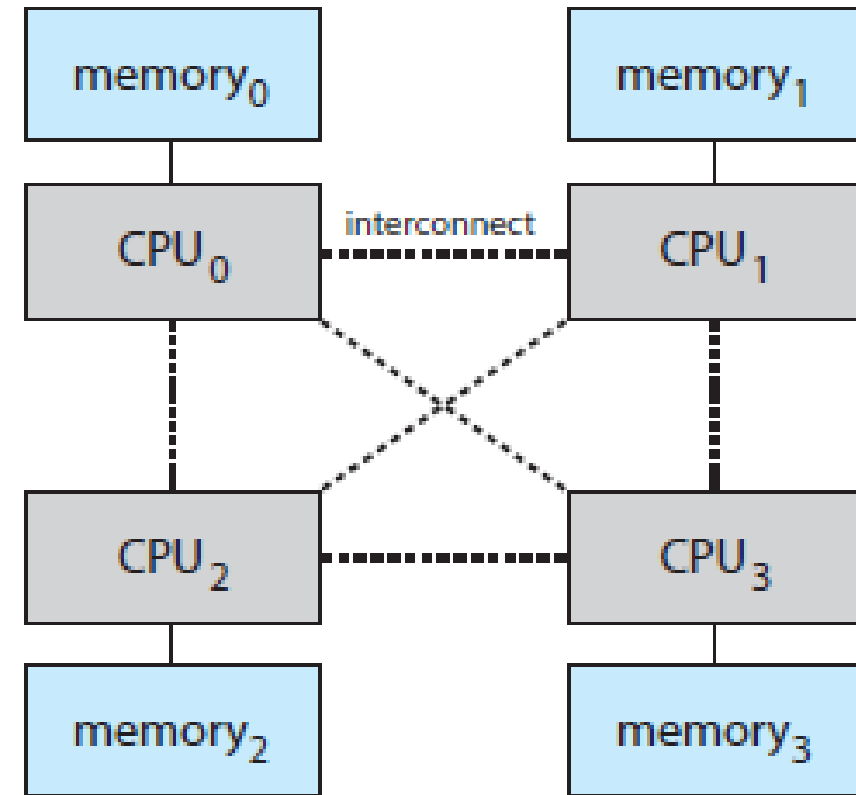
Multi-Processor Systems

An alternate approach,

The CPUs are connected by a **shared system interconnect**, so that all CPUs share one physical address space. This approach—known as **non-uniform memory access**

Advantage: when a CPU accesses its local memory, not only is it fast, but there is also no contention over the system interconnect.

Drawback: increased latency when a CPU must access remote memory



Clustered Systems

Like multiprocessor systems, but multiple systems working together

Usually sharing storage via a **storage-area network (SAN)**

Provides a **high-availability** service which survives failures

- **Asymmetric clustering** has one machine in hot-standby mode
- **Symmetric clustering** has multiple nodes running applications, monitoring each other

Some clusters are for **high-performance computing (HPC)**

- Applications must be written to use **parallelization**

Some have **distributed lock manager (DLM)** to avoid conflicting operations

Clustered Systems

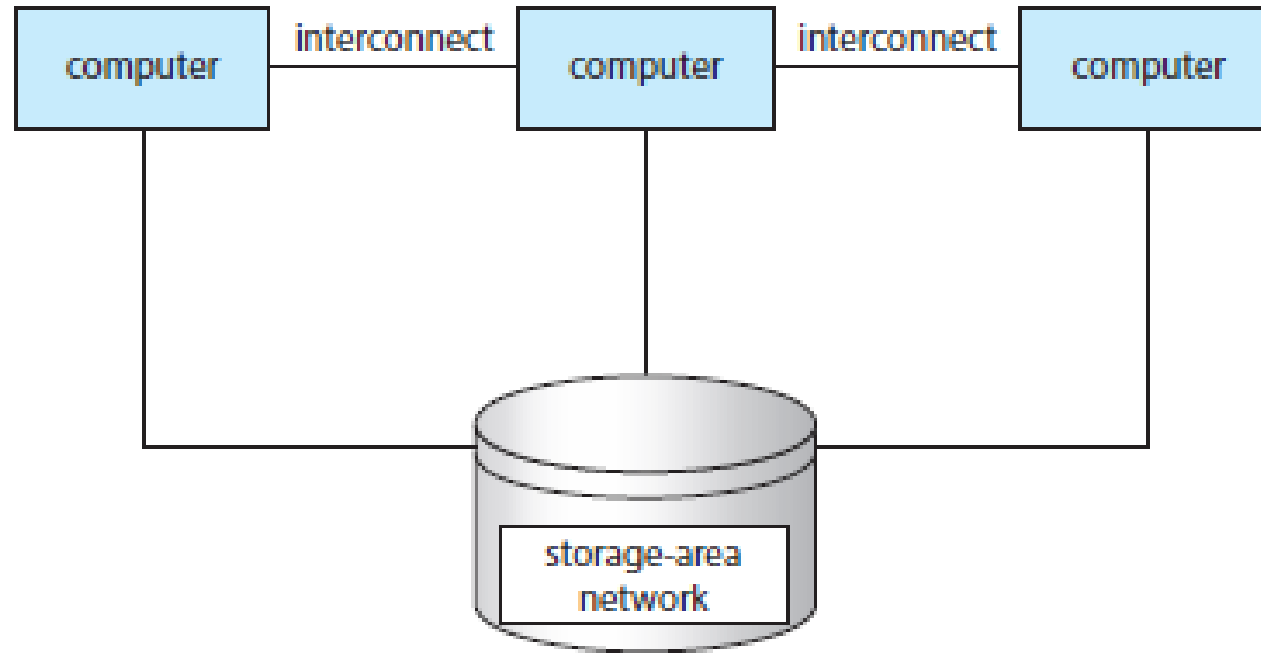


Figure General structure of a clustered system.

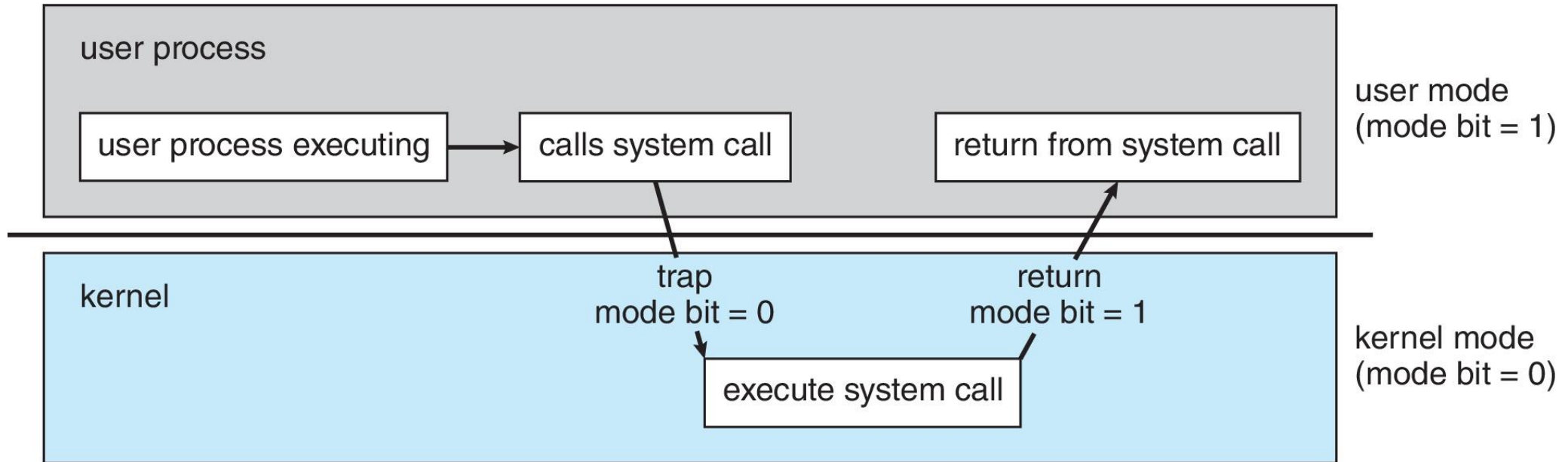
Operating-System Operations

- Bootstrap program – simple code to initialize the system, load the kernel
- Kernel loads
- Starts **system daemons** (services provided outside of the kernel)
- Kernel **interrupt driven** (hardware and software)
 - Hardware interrupt by one of the devices
 - Software interrupt (**exception** or **trap**):
 - Software error (e.g., division by zero)
 - Request for operating system service – **system call**
 - Other process problems include infinite loop, processes modifying each other or the operating system

Dual-mode Operation

- **Dual-mode** operation allows OS to protect itself and other system components
 - **User mode** and **kernel mode**
- **Mode bit** provided by hardware
 - Provides ability to distinguish when system is running user code or kernel code.
 - When a user is running $\Rightarrow$ mode bit is "user"
 - When kernel code is executing $\Rightarrow$ mode bit is "kernel"
- How do we guarantee that user does not explicitly set the mode bit to "kernel"?
 - System call changes mode to kernel, return from call resets it to user
- Some instructions designated as **privileged**, only executable in kernel mode

Transition from User to Kernel Mode



Timer

- Timer to prevent infinite loop (or process hogging resources)
 - Timer is set to interrupt the computer after some time period
 - Keep a counter that is decremented by the physical clock
 - Operating system set the counter (privileged instruction)
 - When counter zero generate an interrupt
 - Set up before scheduling process to regain control or terminate program that exceeds allotted time